Dynamic Programming

Genya Ishigaki

Dynamic Programming

- Dynamic Programming (DP) refers to a collection of algorithms that can be used to compute optimal policies given a perfect model of the environment
- Classical DP algorithms are of limited utility in reinforcement learning because of
  - Their assumption of a perfect model
  - Their great computational expense
- However, DP provides an essential foundation for the understanding RL algorithms we will study

Dynamic Programming

- Main idea: the use of value functions to organize and structure the search for good policies
- [From the last lecture] We can easily obtain optimal policies once we have found the optimal value functions
  - Therefore, we will find the optimal value function through DP

$$
\begin{aligned}
v_*(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \max_a \sum_{s', r} p(s', r \mid s, a) \Big[r + \gamma v_*(s') \Big],
\end{aligned}
$$

Policy Evaluation (Prediction)

- The prediction problem (or policy evaluation) is to compute the state value function for a given policy
- Remember the Bellman Expectation equation:

$$v_\pi(s) \doteq \sum_a \pi(a|s) \sum_{s',r} p(s', r \,|\, s, a) \Big[r + \gamma v_\pi(s') \Big]$$

- By changing a policy, the value of states will change
- In the prediction problem, we fix a policy and compute the state value

Iterative Policy Evaluation [Key Idea]

1. Collect tentative state values of all successor states of a state s
2. Weight the tentative state values by transition x policy probabilities
3. Update the value of state s based on the Bellman expectation equation

$$
\begin{aligned}
v_{k+1}(s) &\doteq \mathbb{E}_{\pi}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(s', r \mid s, a) \Big[r + \gamma v_k(s') \Big]
\end{aligned}
$$

Iterative Policy Evaluation

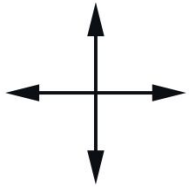
Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π, the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\mathit{terminal})$ to 0

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r \,|\, s, a)\big[r + \gamma V(s')\big]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

An Example: 4x4 Gridworld

- There are four actions possible in each state: up, down, right, left
  - Deterministically cause the corresponding state transitions
  - Actions that would take the agent out of the grid leave the state unchanged
- All transitions until the terminal state (shaded in the figure) is reached
- An undiscounted, episodic task
- On all transitions, the agent gets -1 reward



actions

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

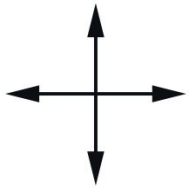
$R_t = -1$
on all transitions

An Example: 4x4 Gridworld

- The Prediction problem given a random policy (all actions equally likely)
- Compute

$$v_\pi(s_5)$$

$$v_\pi(s_9)$$



actions

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$$R_t = -1$$

on all transitions

Milestone Questions

v_k for the random policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

- Compute the next step

An Example: 4x4 Gridworld

v_k for the random policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

The Policy Evaluation to Policy Improvement

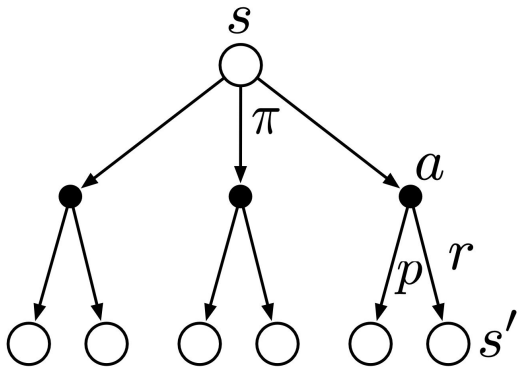
- Goal: Find an optimal policy
- However, the prediction problem only examine the value of states given a policy and does NOT modify the policy
- We need a separate policy improvement algorithm

A Policy Improvement Algorithm

- Consider selecting a in s and thereafter following the existing policy π

$$\begin{aligned} q_\pi(s,a) &\doteq \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma v_\pi(s')\Big]. \end{aligned}$$

- Is $q_\pi(s,a)$ greater than $v_\pi(s)$?
  - If so, it's always better to select a at s



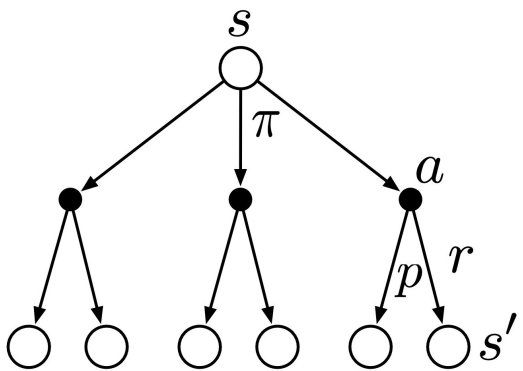
The Policy Improvement Theorem

- Let π and π' be any pair of deterministic policies such that, for all states s,

$$q_\pi(s, \pi'(s)) \geq v_\pi(s)$$

- Then, the policy π' must be as good as, or better than π, for all states s,

$$v_{\pi'}(s) \geq v_\pi(s)$$

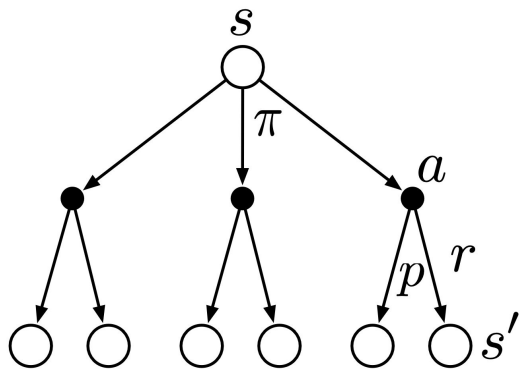


The Policy Improvement Theorem

- The theorem can be applied to the case where
  - an original deterministic policy, π, and a changed policy, π', that is identical to π except that $\pi'(s) = a \neq \pi(s)$
  - This is what we wanted to check! (i.e., Choosing a specific action a from state s is always better than the average performance (state value) of the state s?)

The Policy Improvement Theorem

$$
\begin{aligned}
v_\pi(s) &\leq q_\pi(s, \pi'(s)) \\
&= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s]
\end{aligned}
$$



The Policy Improvement Theorem

$$
\begin{aligned}
v_\pi(s) &\le q_\pi(s, \pi'(s)) \\
&= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\
&\le \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma \mathbb{E}[R_{t+2} + \gamma v_\pi(S_{t+2}) | S_{t+1}, A_{t+1} = \pi'(S_{t+1})] \mid S_t = s] \\
&= \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s\right] \\
&\le \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) \mid S_t = s\right] \\
&\vdots \\
&\le \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \mid S_t = s\right] \\
&= v_{\pi'}(s).
\end{aligned}
$$

How the condition will be realized?

- The condition of the policy improvement theorem

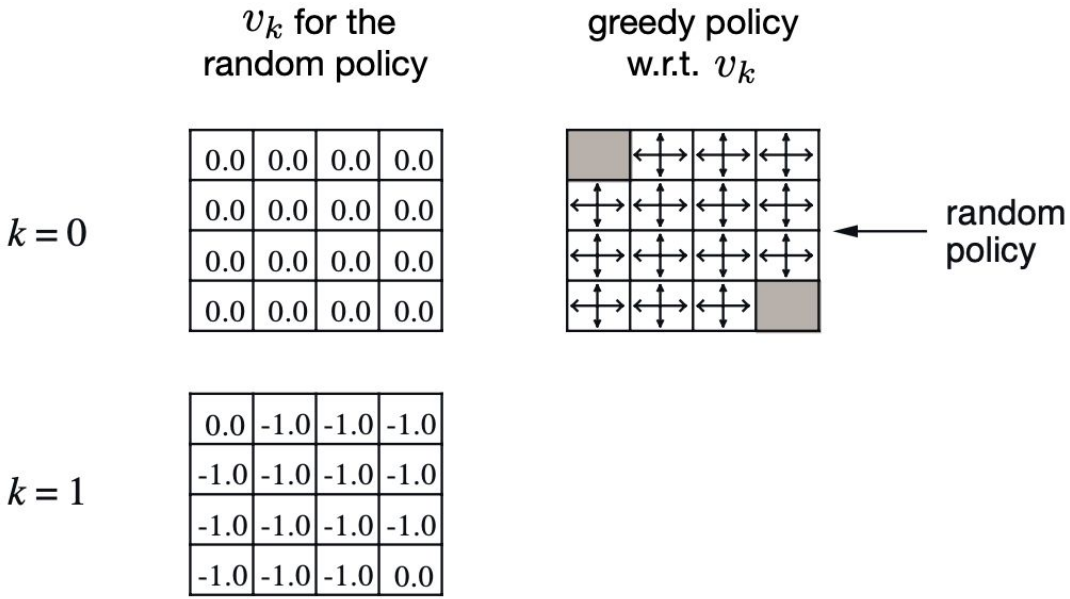
$$q_\pi(s, \pi'(s)) \geq v_\pi(s)$$

- Create the new policy π' with the greedy policy:

$$\begin{aligned}
\pi'(s) &\doteq \arg\max_a q_\pi(s, a) \\
&= \arg\max_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \arg\max_a \sum_{s', r} p(s', r \mid s, a) \Big[r + \gamma v_\pi(s') \Big],
\end{aligned}$$

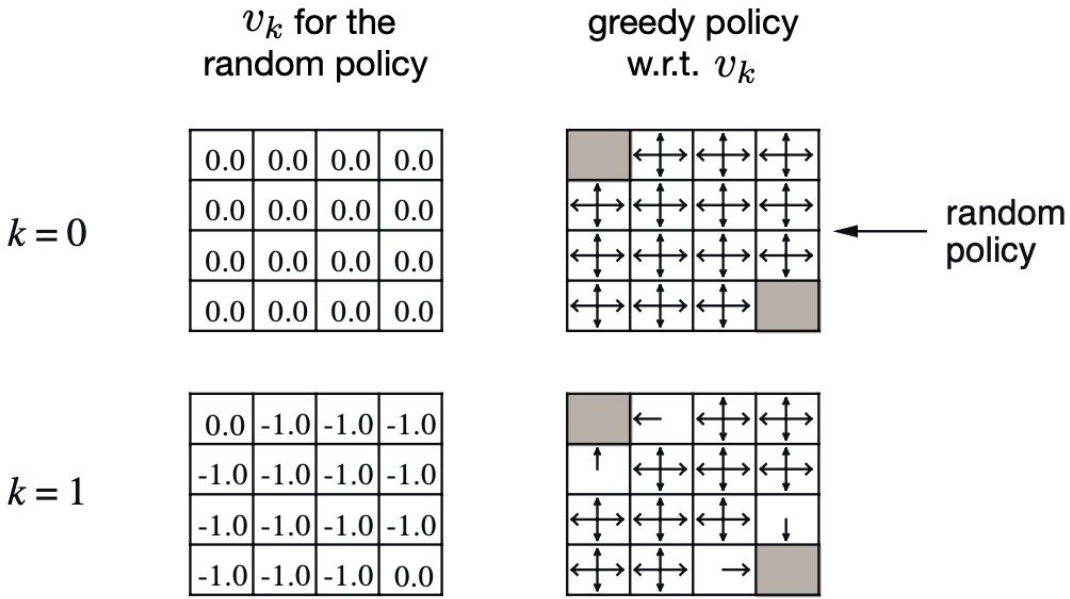
Milestone Questions

- Remember the 4x4 Gridworld
- Update the policy for k = 2 based on the value function estimation at k = 1, using the greedy policy



Milestone Questions

- Remember the 4x4 Gridworld
- Update the policy for k = 2 based on the value function estimation at k = 1, using the greedy policy



Summary so far

- We saw two separate simple algorithms
  - The Policy Evaluation: Obtain the state values, given a policy
  - The Greedy Policy Improvement: Update a policy to improve the state values
- We can repeat the two algorithms until the convergence (Policy Iteration Algorithm)

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

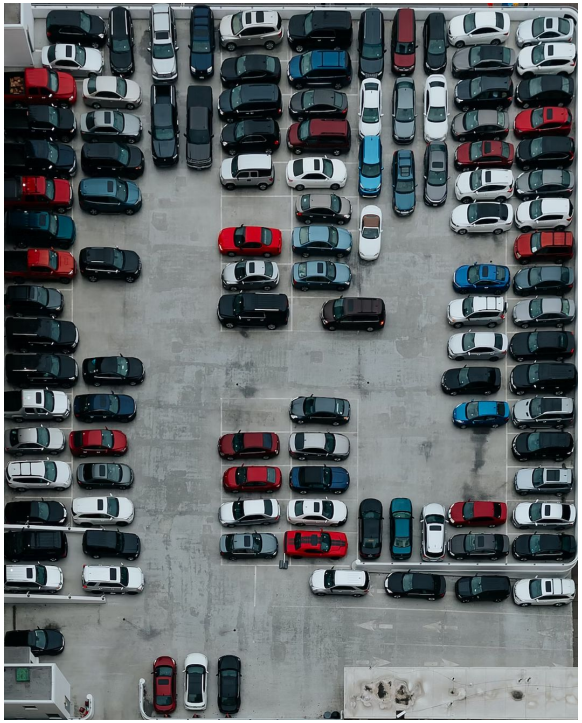
1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\mathit{terminal}) \doteq 0$

2. Policy Evaluation
   Loop:
   $\Delta \leftarrow 0$
   Loop for each $s \in \mathcal{S}$:
   $v \leftarrow V(s)$
   $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s)) \big[r + \gamma V(s')\big]$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   $\mathit{policy\text{-}stable} \leftarrow \mathit{true}$
   For each $s \in \mathcal{S}$:
   $\mathit{old\text{-}action} \leftarrow \pi(s)$
   $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma V(s')\big]$
   If $\mathit{old\text{-}action} \neq \pi(s)$, then $\mathit{policy\text{-}stable} \leftarrow \mathit{false}$
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

An Example: Jack's Car Rental

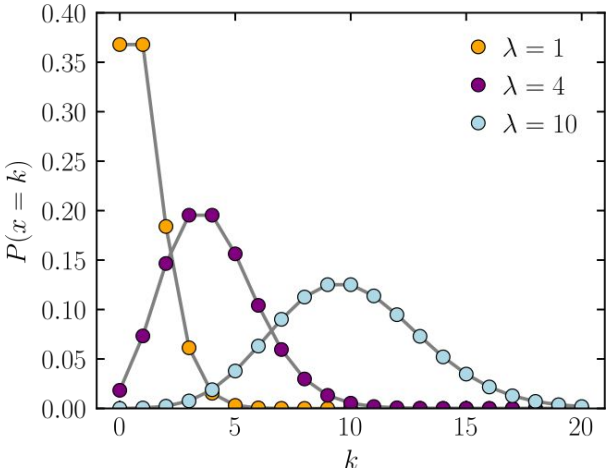
- Jack manages two locations for a nationwide car rental company
  - If a car available, he rents it out (+$10)
  - If not at the location, the business is lost
- Jack can move them between the two locations overnight, at a cost of $2 per car moved
- Some assumptions to simplify the problem
  - The number of cars requested and returned at each location are Poisson random variables
  - There can be no more than 20 cars at each
- What is an optimal number of cars to move, given the number of cars available at each location?

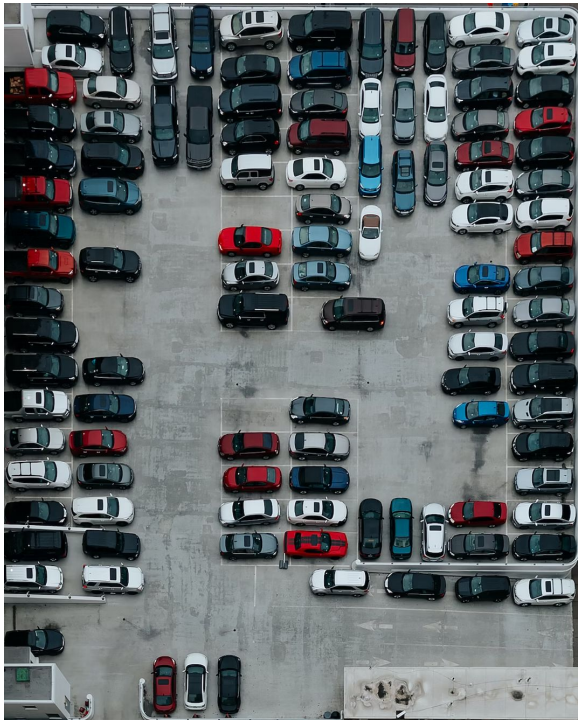


An Example: Jack's Car Rental

The number of cars requested and returned at each location are Poisson random variables

- Location 1: Request Mean 3, Return Mean 3
- Location 2: Request Mean 4, Return Mean 2



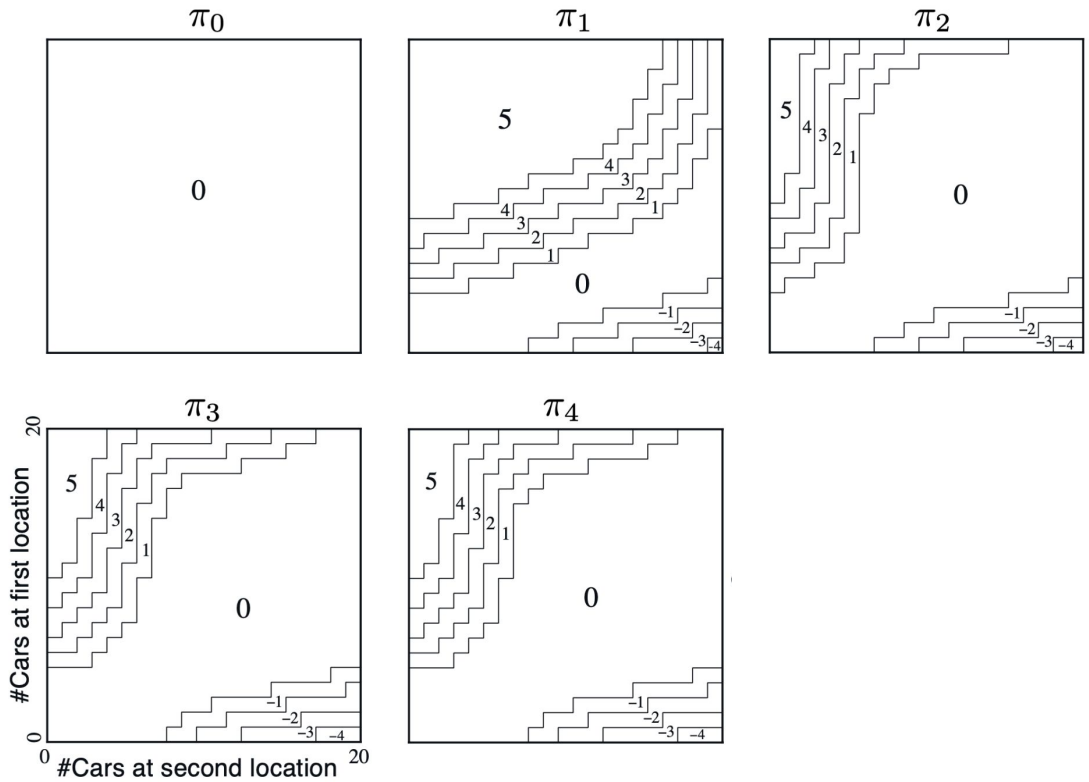


Milestone Questions

- Any reasoning?

Milestone Questions

- After policy iteration rounds



A Drawback of Policy Iteration

2. Policy Evaluation
   Loop:
      $\Delta \leftarrow 0$
      Loop for each $s \in \mathcal{S}$:
         $v \leftarrow V(s)$
         $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))\big[r + \gamma V(s')\big]$
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   $policy\text{-}stable \leftarrow true$
   For each $s \in \mathcal{S}$:
      $old\text{-}action \leftarrow \pi(s)$
      $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
      If $old\text{-}action \neq \pi(s)$, then $policy\text{-}stable \leftarrow false$
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Iterate the entire state space

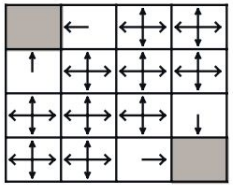
This process may update one or very few actions

A Drawback of Policy Iteration: An Example

- Remember the 4x4 Grid world example
- While the state values change in k = 2, 3, …, 10, the policy did not change
- Computing the exact state values is computationally expensive

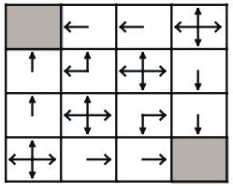
$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0



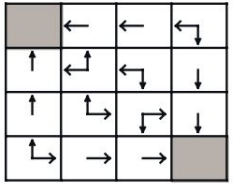
$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0



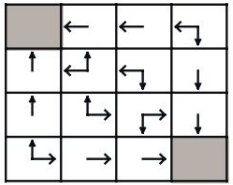
$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0



$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0



optimal policy

Value Iteration

- The policy evaluation step of policy iteration can be truncated in several ways without losing the convergence guarantees
- An idea: Policy evaluation is stopped after just one sweep (one update of each state)
- Value iteration combines, in each of its sweeps, one sweep of policy evaluation and one sweep of policy improvement

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(terminal) \doteq 0$

2. Policy Evaluation
   Loop:
      $\Delta \leftarrow 0$
      Loop for each $s \in \mathcal{S}$:
         $v \leftarrow V(s)$
         $V(s) \leftarrow \sum_{s',r} p(s', r|s, \pi(s))\big[r + \gamma V(s')\big]$
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
      old-action $\leftarrow \pi(s)$
      $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r|s, a)\big[r + \gamma V(s')\big]$
      If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow false$
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Value Iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

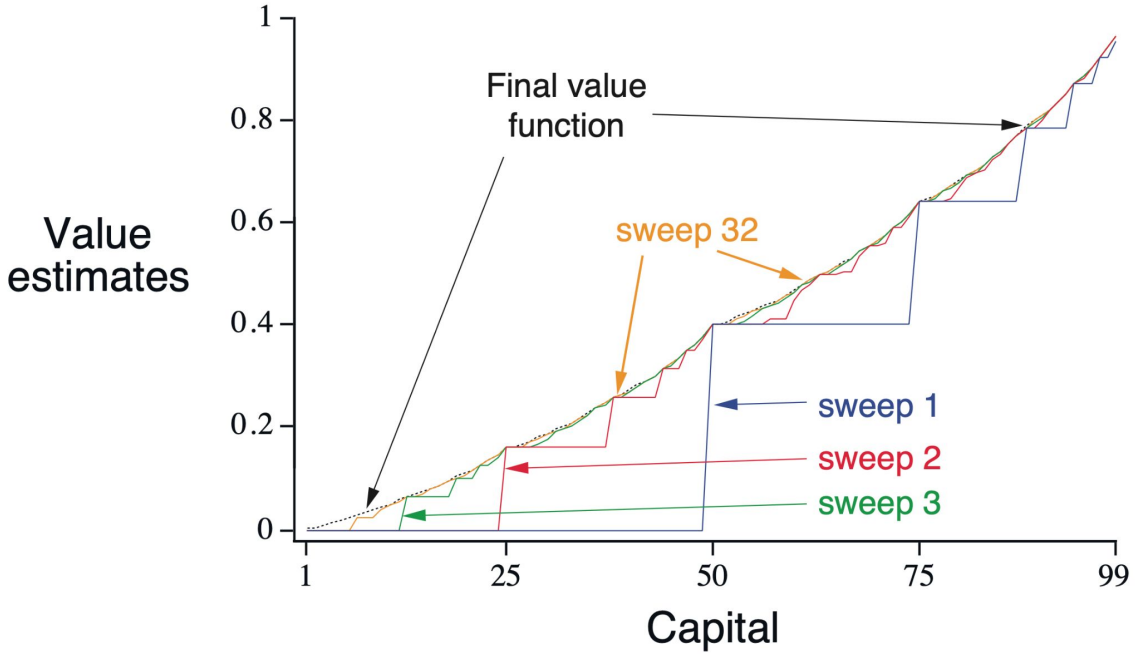
Output a deterministic policy, $\pi \approx \pi_*$, such that
  $\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

An Example: Gambler's Problem

- A gambler makes bets on the outcomes of a sequence of coin flips
  - If the coin comes up **heads**, he wins as many dollars as he has staked on that flip
  - if it is **tails**, he loses his stake
- The game ends when the gambler
  - wins by reaching his goal of $100 or
  - loses by running out of money
- The reward is
  - Zero on all transitions except those on which the gambler reaches his goal
  - When the goal reached, +1

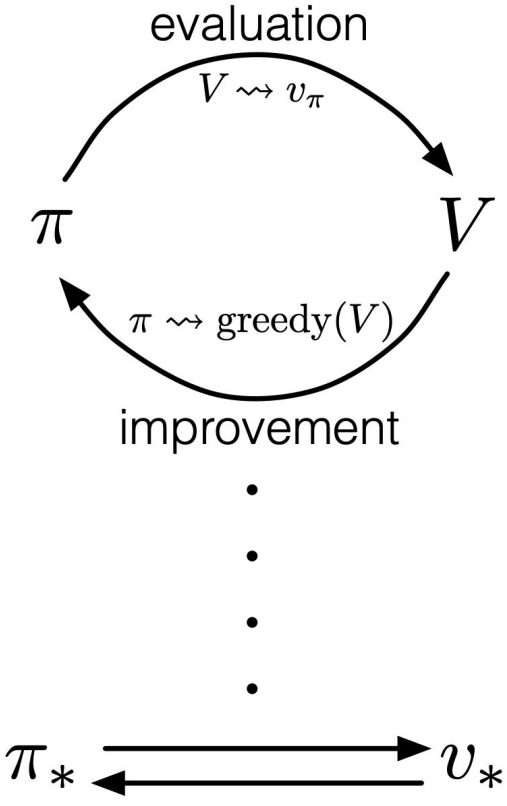
An Example: Gambler's Problem

- Assume a coin with P(head) = 0.4
- A true value function shows the probability of winning from a given state (capital) because the reward is 1 when winning



Summary: Generalized Policy Iteration (GPI)

- The general idea of letting policy-evaluation and policy-improvement processes interact, independent of the granularity and other details of the two processes
- RL algorithms (called value-based RL) follow the same format
  - The policy always being improved with respect to the value function
  - The value function always being driven toward the value function for the policy



Summary: Generalized Policy Iteration (GPI)

- The evaluation and improvement processes can be viewed as both competing and cooperating
  - Making the policy greedy with respect to the value function typically makes the value function incorrect for the changed policy
  - Making the value function consistent with the policy typically causes that policy no longer to be greedy
- In the long run, however, these two processes interact to find a single joint solution: the optimal value function and an optimal policy.